

Adding Tools to Your LangChain Agent: Search, Image Generation, Image Analysis, and Voice

A step-by-step guide to wiring four capabilities into the `genai_app` project's agent — web search (Tavily), image generation (Pollinations), image analysis (Gemini Vision), and text-to-speech (YarnGPT). Follows the same `core/tools.py` → `core/agents.py` → `pages/` separation used throughout the LangChain guide.

Setup

```
pip install langchain-tavily langchain-google-genai requests
# .env
TAVILY_API_KEY=your-tavily-key          # free tier at tavily.com — no card required
GOOGLE_API_KEY=your-gemini-key
```

1. Web Search Tool (Tavily)

Tavily is the standard, LangChain-maintained search tool — free tier, purpose-built for feeding results to an LLM (cleaner than scraping raw search engine HTML yourself).

```
# core/tools.py
from langchain_tavily import TavilySearch

web_search = TavilySearch(
    max_results=5,
    topic="general",
)
```

That's it — `TavilySearch` is already a fully-formed LangChain tool, no `@tool` wrapper needed. Test it directly:

```
result = web_search.invoke({"query": "latest news on AI regulation in Africa"})
print(result)
```

2. Image Generation Tool (Pollinations)

Free, no API key required. Wrapped as a proper `@tool` so an agent can decide when to call it:

```
# core/tools.py (continued)
import os
import requests
import urllib.parse
from langchain_core.tools import tool
```

```
@tool
def generate_image(prompt: str) -> str:
    """Generate an image from a text description and return the file path.
    Use this when the user asks you to create, draw, or visualize something.

    Args:
        prompt: A clear, descriptive prompt of the image to generate.
    """
    encoded = urllib.parse.quote(prompt)
    url = f"https://image.pollinations.ai/prompt/{encoded}?
width=1024&height=1024&nologo=true"
    response = requests.get(url, timeout=60)
    response.raise_for_status()

    os.makedirs("generated_images", exist_ok=True)
    path = f"generated_images/{abs(hash(prompt))}.png"
    with open(path, "wb") as f:
        f.write(response.content)
    return path
```

3. Image Analysis Tool (Gemini Vision)

Gemini takes an image and text together in one multimodal message. This tool accepts a local file path (from an upload, or from `generate_image` above) plus a question, and returns Gemini's analysis.

```
# core/tools.py (continued)
import base64
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain_core.messages import HumanMessage

vision_model = ChatGoogleGenerativeAI(model="gemini-2.5-flash")

@tool
def analyze_image(image_path: str, question: str) -> str:
    """Analyze an image and answer a question about it.
    Use this when the user uploads an image and asks what's in it,
    or asks you to describe, identify, or extract information from an image.

    Args:
        image_path: Local file path to the image.
        question: What to determine about the image.
    """
    with open(image_path, "rb") as f:
        image_bytes = f.read()
    b64_image = base64.b64encode(image_bytes).decode("utf-8")

    message = HumanMessage(content=[
        {"type": "text", "text": question},
        {"type": "image_url", "image_url": {"url": f"data:image/jpeg;base64,{b64_image}"}}],
    ])
    response = vision_model.invoke([message])
```

```
return response.content
```

This uses a dedicated Gemini client rather than your main agent's model, so image analysis works even if your primary model (Claude, Groq, whichever) has no vision support — a deliberate design choice, not an accident.

4. Voice Output (YarnGPT) — an important honesty check first

YarnGPT does not have a hosted API. It's a Hugging Face model you run yourself — the model card's own "Future Improvements" section literally lists "wrap the model around an API endpoint" as something not yet done. There's no `requests.get()` equivalent here like Pollinations. Two real paths forward:

Option A — Run it locally/in Colab and call it as a plain function (not a hosted API)

```
# core/voice.py
# One-time setup (run once per Colab session):
# !git clone https://github.com/saheedniyi02/yarngpt.git
# !pip install outetts==0.2.3 uroman
# !wget https://huggingface.co/novateur/WavTokenizer-medium-speech-75token/resolve/main/wavtokenizer_mediumdata_frame75_3s_nq1_code4096_dim512_kmeans200_attn.yaml

import torch
import torchaudio
from transformers import AutoModelForCausalLM, AutoTokenizer
from yarngpt.audiotokenizer import AudioTokenizerV2

_model = None
_tokenizer = None
_audio_tokenizer = None

def _load_yarngpt():
    global _model, _tokenizer, _audio_tokenizer
    if _model is None:
        tokenizer_path = "saheedniyi/YarnGPT2"
        _audio_tokenizer = AudioTokenizerV2(
            tokenizer_path,
            "wavtokenizer_large_speech_320_24k.ckpt",
            "wavtokenizer_mediumdata_frame75_3s_nq1_code4096_dim512_kmeans200_attn.yaml",
        )
        _model = AutoModelForCausalLM.from_pretrained(tokenizer_path,
            torch_dtype="auto").to("cuda")
        _tokenizer = AutoTokenizer.from_pretrained(tokenizer_path)
    return _model, _tokenizer, _audio_tokenizer

def text_to_speech(text: str, voice: str = "idera") -> str:
    """Generate Nigerian-accented English speech from text. Returns path to a .wav file."""
    model, tokenizer, audio_tokenizer = _load_yarngpt()
    prompt = audio_tokenizer.create_prompt(text, voice)
```

```

input_ids = audio_tokenizer.tokenize_prompt(prompt)
output = model.generate(input_ids=input_ids, max_length=4000, temperature=0.1,
repetition_penalty=1.1)
codes = audio_tokenizer.get_codes(output)
audio = audio_tokenizer.get_audio(codes)
path = "generated_audio/output.wav"
torchaudio.save(path, audio, sample_rate=24000)
return path

```

This needs a GPU runtime (Colab's free T4 works) and requires the setup commands run once at the top of your notebook/session — it will **not** run inside a normal Streamlit deployment without that same setup, and it's meaningfully heavier than the other three tools (a real ~360M-parameter model loaded into memory, not an API call).

Option B — Use it inline as a tool once loaded

```

# core/tools.py (continued)
from core.voice import text_to_speech as _yarn_gpt_tts

@tool
def speak_nigerian_english(text: str) -> str:
    """Convert text to Nigerian-accented English speech. Returns a path to a .wav audio
file.
    Use this when the user asks you to read something aloud or generate audio/voice
output."""
    return _yarn_gpt_tts(text)

```

Practical recommendation: if you specifically need a genuinely hosted, zero-setup TTS API (not self-hosted), consider ElevenLabs (has a free tier, but time-limited rather than permanent) as an alternative — worth deciding whether the Nigerian-accent authenticity YarnGPT offers is worth the self-hosting cost for your use case, versus a hosted alternative that trades the accent for zero infrastructure burden.

5. Assemble the Agent

```

# core/agents.py
from langchain.agents import create_agent
from core.tools import web_search, generate_image, analyze_image, speak_nigerian_english
from core.models import get_model

def build_multimodal_agent():
    return create_agent(
        model=get_model(),
        tools=[web_search, generate_image, analyze_image, speak_nigerian_english],
        system_prompt=(
            "You are a helpful assistant with these capabilities: "
            "web_search for current information, generate_image to create images, "
            "analyze_image to describe or answer questions about an uploaded image, "
            "and speak_nigerian_english to produce spoken audio of text. "
            "Use the tool that matches what the user is actually asking for."
        ),
    ),

```

)

6. The Streamlit Page

```
# pages/8_Multimodal_Assistant.py
import streamlit as st
from core.agents import build_multimodal_agent

st.set_page_config(page_title="Multimodal Assistant", page_icon="🌱")

agent = build_multimodal_agent()

uploaded_image = st.file_uploader("Upload an image (optional)", type=["png", "jpg", "jpeg"])
if uploaded_image:
    with open(f"uploads/{uploaded_image.name}", "wb") as f:
        f.write(uploaded_image.read())
    st.image(uploaded_image, width=300)

prompt = st.chat_input("Ask, search, generate, or analyze")
if prompt:
    with st.chat_message("user"):
        st.markdown(prompt)
    with st.chat_message("assistant"):
        try:
            full_prompt = prompt
            if uploaded_image:
                full_prompt += f" (Image available at: uploads/{uploaded_image.name})"
            result = agent.invoke({"messages": [{"role": "user", "content": full_prompt}]})
            st.markdown(result["messages"][-1].content)
        except Exception as e:
            st.error("Something went wrong processing that request.")
```

Quick Sanity Checklist

Tool	Setup burden	Notes
Web search (Tavily)	Low — API key only	Works immediately
Image generation (Pollinations)	None — no key needed	Works immediately, no SLA
Image analysis (Gemini)	Low — API key only	Works immediately
Voice (YarnGPT)	High — GPU + local model load	Test standalone in a Colab cell before wiring into the agent; no hosted API exists

Test the voice tool in isolation first — it's the most likely of the four to need debugging, since it's a real model running in-process rather than a simple API call.